

Bagaimana CNN (*Convolutional Neural Network*) bekerja?

Fajar Adie Santosa (@debug.stack)

2026

Contents

Panduan Notasi	3
Pendahuluan	8
STEP 1: Input	8
STEP 2: Convolution dan ReLU	10
Convolution	10
ReLU	11
Image-to-Column	11
Conv Forward	13
STEP 3: Pooling	15
STEP 4: Flatten	17
STEP 5: Fully Connected	18
STEP 6: Softmax	19
STEP 7: Loss Function	19
STEP 8: Backpropagation	21
Backward pada Fully Connected dan ReLU	21
Backward pada Flatten dan Max Pooling	22
Backward pada Convolution	23
STEP 9: Gradient Descent	25
STEP 10: Evaluasi Model	27
Hasil Training	28

Panduan Notasi

Sebelum masuk ke pembahasan, kita kenali dulu simbol yang akan sering muncul. Tidak perlu dihafalkan semuanya; kalian bisa kembali ke bagian ini ketika menemukan notasi yang belum dipahami. Beberapa huruf memiliki arti berbeda tergantung konteks rumus atau kode, jadi perhatikan juga penjelasan di sekitarnya.

Notasi	Arti dan cara membacanya
Data dan ukuran tensor	
x, X	Input model. Pada projek ini berupa nilai piksel atau kumpulan gambar MNIST. Huruf besar biasanya dipakai untuk kumpulan data atau matriks, tetapi kode juga memakai <code>x</code> untuk tensor input.
x_{norm}	Nilai piksel setelah normalisasi, yaitu nilai piksel awal dibagi 255.
y, y_i	Label yang benar; y_i berarti label sampel ke- i . Dalam notebook, label berupa bilangan bulat 0 sampai 9.
$y_{i,c}$	Label dalam bentuk indikator kelas: bernilai 1 jika sampel ke- i termasuk kelas c , dan 0 jika tidak. Bentuk ini dipakai pada rumus loss.
$p, p_{i,c}$	Probabilitas hasil softmax. $p_{i,c}$ berarti probabilitas sampel ke- i masuk ke kelas c .
N, n	Jumlah sampel yang sedang diproses. Pada perhitungan loss satu batch, nilainya adalah jumlah sampel dalam batch tersebut. Pada rumus softmax, n menyatakan jumlah kelas.
C, c	Pada convolution, C menyatakan jumlah channel. Pada rumus loss, C adalah jumlah kelas dan c adalah indeks kelas.
H, W	Tinggi dan lebar gambar atau feature map ketika membahas ukuran tensor. Dalam kode, lebar juga ditulis <code>W_input</code> atau <code>width</code> .
(N, C, H, W)	Urutan shape tensor gambar: jumlah sampel, channel, tinggi, lalu lebar. Contoh $(512, 1, 28, 28)$ berarti 512 gambar grayscale berukuran 28×28 .

Notasi	Arti dan cara membacanya
$H_{\text{out}}, W_{\text{out}}$	Tinggi dan lebar hasil convolution atau pooling. Dalam kode ditulis <code>h_out</code> , <code>w_out</code> , atau menggunakan huruf besar.
K_h, K_w, K	Tinggi dan lebar kernel. K dipakai untuk ukuran sisi jika kernel berbentuk persegi; dalam kode ditulis <code>KH</code> , <code>KW</code> , <code>kh</code> , atau <code>kw</code> .
S, s, stride	Banyaknya langkah pergeseran kernel atau window pooling.
<code>pad</code>	Jumlah piksel padding pada tiap sisi input. Convolution di notebook menggunakan <code>pad=0</code> .
F, f	Pada convolution, jumlah filter. Pada rumus pooling, f berarti ukuran window. Variabel F pada <code>forward_cnn</code> berarti hasil flatten.
n_h, n_w, n_c	Notasi lain untuk tinggi, lebar, dan jumlah channel pada pembahasan pooling.

Notasi	Arti dan cara membacanya
Parameter, aktivasi, dan gradien	
$W, W^{(l)}$	Bobot model atau bobot pada layer ke- l . Berbeda dari W yang berarti lebar pada shape tensor; artinya mengikuti konteks.
$b, b^{(l)}$	Bias, yaitu parameter yang ditambahkan pada hasil perkalian input dan bobot.
Z, z	Nilai sebelum fungsi aktivasi. Pada output terakhir, nilai ini disebut logits.
$A, A^{(l)}$	Hasil aktivasi pada layer ke- l . $A^{(0)} = X$ berarti input dianggap sebagai aktivasi awal.
$\sigma, \sigma^{(l)}$	Fungsi aktivasi. Pada pembahasan softmax, $\sigma(z_i)$ menyatakan probabilitas kelas ke- i .
$l, (l)$	Indeks layer. Superskrip dalam tanda kurung, seperti $W^{(l)}$, menunjukkan layer, bukan pangkat.
L	Pada arsitektur fully connected, jumlah total layer. Pada pembahasan loss dan optimasi, L berarti nilai loss.
θ	Sebutan umum untuk parameter yang dipelajari, misalnya bobot W atau bias b .

Notasi	Arti dan cara membacanya
η, lr	Learning rate, yaitu besar langkah perubahan parameter saat update.
$\frac{\partial L}{\partial x}$	Turunan parsial loss terhadap x : menunjukkan bagaimana loss berubah saat x berubah, dengan variabel bebas lain dianggap tetap.
$\nabla_{\theta} L$	Gradien loss terhadap parameter θ , yaitu kumpulan turunan parsial untuk seluruh elemen parameter tersebut.
dZ, dA, dW, db, dX	Singkatan gradien loss terhadap masing-masing variabel. Contoh $dW = \partial L / \partial W$, bukan nilai bobot yang baru.
$\text{dz3}, \text{dW1}, \text{db2}$	Nama gradien pada kode. Angka mengikuti penamaan operasi/parameter notebook: 1 untuk convolution, 2 untuk dense pertama, dan 3 untuk output dense.
dout	Gradien terhadap output sebuah operasi yang diterima dari tahap backward sebelumnya.
$\text{cols}, W_{\text{col}}$	Matriks kumpulan patch hasil <code>im2col</code> dan matriks bobot kernel yang sudah diratakan. Subskrip <code>col</code> menandai bentuk matriks untuk perhitungan convolution.
$dZ_{\text{col}}, d\text{cols}$	Gradien output convolution dalam bentuk matriks dan gradien terhadap matriks patch input.
$\text{cache}, \text{params}$	Cache menyimpan nilai forward yang diperlukan saat backward. Params menyimpan kumpulan bobot dan bias model.

Notasi	Arti dan cara membacanya
Operator matematika dan notasi NumPy	
$i, j, \dots$	Indeks untuk menunjuk elemen, sampel, atau posisi. Tanda $\dots$ berarti urutan berlanjut. Indeks rumus sering mulai dari 1, sedangkan indeks array Python mulai dari 0.
$\sum_{i=1}^N$	Jumlahkan semua suku dari indeks $i = 1$ sampai $i = N$. Dua simbol penjumlahan berarti menjumlahkan pada dua indeks, misalnya sampel dan kelas.

Notasi	Arti dan cara membacanya
$\frac{a}{b}, \cdot, \times$	Pembagian dan perkalian bilangan. Pada ukuran 28×28 , tanda $\times$ dibaca “kali” untuk tinggi dan lebar.
$AW, A @ W$	Perkalian matriks. Jumlah kolom matriks kiri harus sama dengan jumlah baris matriks kanan.
$\odot, *$	Perkalian elemen demi elemen. Pada array NumPy, $*$ mengikuti aturan broadcasting; pada bilangan biasa, ini perkalian biasa.
$A^T, A.T$	Transpose: menukar baris dan kolom matriks. Untuk tensor, <code>transpose(...)</code> mengatur ulang urutan sumbu.
$e^z, \exp(z)$	Fungsi eksponensial dengan bilangan dasar $e \approx 2.718$. Dalam NumPy ditulis <code>np.exp(z)</code> .
$\log(p)$	Logaritma natural, yaitu logaritma berbasis e . Dalam NumPy ditulis <code>np.log(p)</code> .
$\mathbb{I}(Z > 0)$	Fungsi indikator atau mask: 1 jika elemen Z positif dan 0 jika tidak. Pada kode ditulis <code>Z > 0</code> .
<code>max, argmax</code>	Max mengambil nilai terbesar; argmax mengambil indeks nilai terbesar. Indeks ini digunakan untuk memilih kelas prediksi.
$\in, [0, 1]$	Simbol $\in$ dibaca “anggota dari”. Interval $[0, 1]$ mencakup seluruh nilai dari 0 sampai 1, termasuk kedua batasnya.
$\leftarrow, -=, +=$	Panah kiri berarti memperbarui nilai. <code>W -= lr * dW</code> mengurangi bobot dengan langkah gradien; <code>+=</code> menambahkan nilai ke isi variabel yang sudah ada.
$\rightarrow$	Arah aliran proses, misalnya output dense menuju ReLU saat backward.
<code>//</code>	Pembagian dengan pembulatan ke bawah. Contoh <code>5 // 2</code> menghasilkan 2; digunakan untuk menghitung ukuran output.
<code>shape, reshape</code>	Shape adalah ukuran tiap sumbu array. Reshape mengubah susunan ukuran tersebut dengan jumlah elemen tetap; -1 meminta NumPy menghitung ukuran sumbu itu.
<code>:</code> , <code>None</code>	Dalam indeks array, <code>:</code> mengambil seluruh elemen pada sumbu itu. <code>None</code> menyisipkan sumbu baru berukuran 1, misalnya untuk channel grayscale.

Notasi	Arti dan cara membacanya
<code>axis, keepdims</code>	Axis memilih sumbu operasi seperti penjumlahan. <code>keepdims=True</code> mempertahankan sumbu yang direduksi dengan ukuran 1 agar shape mudah dipadankan.
<code>1e-12</code>	Penulisan ilmiah untuk 10^{-12} . Pada kode loss, bilangan kecil ini ditambahkan sebelum logaritma untuk menghindari $\log(0)$.

Sebagai contoh, $dW = A^T dZ$ dibaca: “gradien bobot didapat dari transpose aktivasi dikalikan dengan gradien output”. Dalam NumPy, rumus yang sama ditulis `dW = A.T @ dZ`. Dengan begitu, kita bisa menghubungkan simbol pada rumus dengan operasi yang ada di kode.

Pendahuluan

Model CNN (*Convolutional Neural Network*) adalah salah satu jenis jaringan saraf tiruan yang digunakan untuk pengolahan data seperti gambar atau time series. Di era sekarang ini, membuat model CNN hanya memerlukan import library seperti TensorFlow atau PyTorch dan menulis beberapa baris kode untuk membangun model CNN. Namun, pernahkah berpikir, bagaimana sebenarnya CNN bekerja? mengapa memerlukan parameter dari library ini? dari rasa penasaran tersebut, dilakukan pembuatan ulang CNN dari awal menggunakan Python dan NumPy, tanpa menggunakan library deep learning apapun. Dengan begitu, dapat memahami bagaimana CNN bekerja.

Prasyarat Sebelum Melanjutkan

Dokumen ini membahas CNN sampai ke proses menghitung gradien secara manual. Supaya penjelasannya lebih mudah diikuti, ada dua materi dasar yang sebaiknya dipelajari terlebih dahulu:

- **Kalkulus:** fungsi, turunan, turunan parsial, gradien, dan *chain rule*. Materi ini digunakan saat menghitung seberapa besar perubahan loss terhadap setiap bobot dan bias.
- **Aljabar linear:** skalar, vektor, matriks, tensor, dimensi atau *shape*, transpose, *reshape*, dan perkalian matriks. Materi ini digunakan hampir di seluruh operasi CNN, mulai dari convolution sampai fully connected layer.

Tidak harus menguasai seluruh kalkulus dan aljabar linear. Minimal, pahami mengapa matriks dapat dikalikan dan bagaimana turunan berantai digunakan untuk menelusuri perubahan dari output kembali ke input.

STEP 1: Input

Pada proyek ini, input yang digunakan adalah gambar, menggunakan dataset MNIST (yang didapat dari kaggle). Terus, bagaimana Neural Network (NN) bisa memahami gambar? sepahaman saya, gambar merupakan kumpulan dari pixel, dan pixel merupakan kumpulan dari angka. Namun, angka dari

gambar itu mempresentasikan apa? angka dari gambar itu mempresentasikan intensitas cahaya, dimana semakin besar angka, semakin terang pixel tersebut. Untuk lebih jelasnya, kalian bisa mempelajarinya di youtube ini. Menurut saya, penjelasan di youtube ini sangat mudah dipahami dan ada visual yang membantu untuk memahami konsep CNN ini.

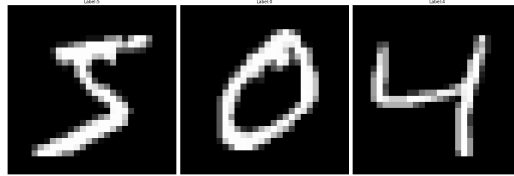


Figure 1: Contoh gambar digit dari dataset MNIST

Karena dataset didapat dari Yann LeCun, maka diperlukan library `idx2numpy` untuk mengubah dataset menjadi numpy array.

```

1 import idx2numpy
2 # for training
3 x_train=idx2numpy.convert_from_file("/kaggle/input/datasets/
   hojjatk/mnist-dataset/train-images.idx3-ubyte")
4 y_train=idx2numpy.convert_from_file("/kaggle/input/datasets/
   hojjatk/mnist-dataset/train-labels.idx1-ubyte")
5
6 # for test
7 x_test=idx2numpy.convert_from_file("/kaggle/input/datasets/
   hojjatk/mnist-dataset/t10k-images.idx3-ubyte")
8 y_test=idx2numpy.convert_from_file("/kaggle/input/datasets/
   hojjatk/mnist-dataset/t10k-labels.idx1-ubyte")

```

Listing 1: Contoh kode untuk mengubah dataset menjadi numpy array

intinya tambahkan juga untuk normalisasi data, karena data gambar memiliki nilai pixel 0-255, sehingga perlu dinormalisasi menjadi 0-1. Normalisasi ini dilakukan dengan membagi nilai pixel dengan 255.0. Dengan begitu, model CNN dapat lebih cepat dan lebih baik dalam belajar dari data gambar. Dengan rumus min-max normalisasi, yaitu:

$$x_{norm} = \frac{x}{255.0}, x \in [0, 1] \quad (1)$$

karena sangat mempengaruhi proses training.

STEP 2: Convolution dan ReLU

Convolution

Sebelum kita membahas lebih dalam tentang convolution, kita perlu memahami arti kernel. Kernel merupakan filter atau matriks kecil yang digunakan untuk mendeteksi fitur. Kernel ini akan melakukan proses konvolusi pada gambar input, dimana kernel akan menggeser posisi pada gambar input dan melakukan perkalian elemen demi elemen antara kernel dan bagian dari gambar input yang sedang diproses.

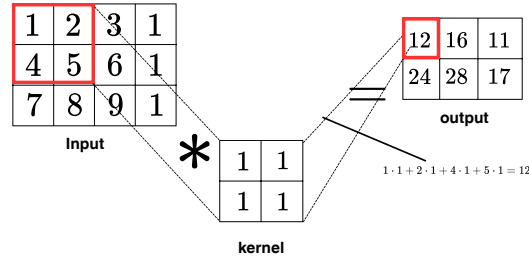


Figure 2: Ilustrasi proses konvolusi pada gambar input dengan kernel

Dari kernel yang bisa digeser posisinya, itu ada nama nya, yaitu stride. Stride merupakan berapa langkah pergeseran kernel pada gambar input. Semakin besar stride, semakin cepat proses konvolusi. Untuk rumus ukuran output dari proses konvolusi:

$$H_{\text{out}} = \frac{(H + 2 * \text{pad} - K_h)}{S} + 1 \quad (2)$$

yang dimana, K adalah panjang kernel, H adalah tinggi input (jumlah baris gambar/feature map).

$$W_{\text{out}} = \frac{(W_{\text{input}} + 2 * \text{pad} - K_w)}{S} + 1 \quad (3)$$

Kemudian, kita lanjut arti konvolusi. Konvolusi merupakan proses perkalian elemen demi elemen antara kernel dan bagian dari gambar input, kemudian hasil perkalian tersebut dijumlahkan dan ditambahkan bias, sehingga menghasilkan output. Output dari proses konvolusi ini disebut feature map. Untuk ilustrasi proses konvolusi bisa dilihat pada gambar 2.

ReLU

ReLU (*Rectified Linear Unit*) merupakan fungsi aktivasi yang digunakan untuk mengubah nilai negatif menjadi nol, dan nilai positif tetap.

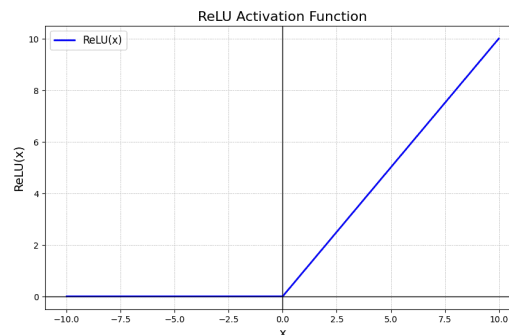


Figure 3: Relu Activation (sumber:geeksforgeeks)

Pada proyek ini kita menggunakan ReLU sebagai fungsi aktivasi, karena berdasarkan penelitian (cari penelitiannya), ReLU lebih cocok untuk menyelesaikan masalah konvolusi. Aktivasi fungsi ReLU ini akan diterapkan pada hidden layer neural network.

Image-to-Column(im2col)

Kalau konvolusi diimplementasikan dengan for-loop biasa (geser kernel piksel demi piksel), prosesnya akan sangat lambat di Python/NumPy, karena NumPy dioptimasi untuk operasi matriks berukuran besar (perkalian matriks/vektorisasi), bukan untuk loop bertingkat. Trik yang umum dipakai adalah im2col (image-to-column): setiap patch gambar yang akan dikalikan dengan kernel diubah menjadi satu baris pada matriks besar, sehingga seluruh proses konvolusi bisa dihitung sebagai satu kali perkalian matriks saja

```
1 def im2col(x, kernel_size, stride=1):
2     """
3     Mengubah setiap patch gambar menjadi satu baris matrix.
4
5     Input:
6         x           : (N, C, H, W)
7         kernel_size : (KH, KW)
8         stride      : langkah pergeseran kernel
9     """
```

```

10     Output:
11         cols           : (N * H_out * W_out, C * KH * KW)
12     """
13
14     N, C, H, W = x.shape
15
16     KH, KW = kernel_size
17
18     # Ukuran output convolution
19     H_out = (H - KH) // stride + 1 # untuk menentukan ukuran
kernel pada height
20     W_out = (W - KW) // stride + 1 # untuk menentukan posisi
kernel pada weight
21
22     # Tempat menyimpan semua patch
23     cols = np.empty(
24         (N, H_out, W_out, C, KH, KW),
25         dtype=x.dtype
26     )
27
28     # Ambil setiap posisi kernel
29     for i in range(KH):
30         for j in range(KW):
31
32             cols[:, :, :, :, i, j] = (
33                 x[
34                     :,
35                     :,
36                     i:i + stride * H_out:stride,
37                     j:j + stride * W_out:stride
38                 ]
39                 .transpose(0, 2, 3, 1)
40             )
41
42     # Ubah semua patch menjadi baris
43     cols = cols.reshape(
44         N * H_out * W_out,
45         C * KH * KW
46     )
47
48     return cols

```

Listing 2: Fungsi `im2col` untuk mengubah patch gambar menjadi baris matriks

Setelah gambar diubah menjadi matriks *cols* berukuran $(N \cdot H_{\text{out}} \cdot W_{\text{out}}, C \cdot$

$K_h \cdot K_w, F$), dan kernel juga diratakan (reshape) menjadi matriks berukuran $(C \cdot K_h \cdot K_w, F)$ dengan F adalah jumlah filter, maka konvolusi cukup dihitung dengan satu kali perkalian matriks : $\text{cols} \odot W_{\text{col}}$

Convolution Forward

Dengan bantuan `im2col`, forward pass dari layer convolution bisa ditulis sebagai berikut: ubah input menjadi cols, kalikan dengan kernel yang sudah diratakan, tambahkan bias, lalu bentuk ulang hasilnya menjadi feature map

```

1  def conv2d_forward(x, W, b, stride=1):
2
3      N, C, H, W_input = x.shape
4      F, _, KH, KW = W.shape
5
6      # Padding
7      x_pad = x
8
9      # Ambil semua patch
10     cols = im2col(
11         x_pad,
12         kernel_size=(KH, KW),
13         stride=stride
14     )
15
16     # Ubah kernel menjadi matrix
17     W_col = W.reshape(F, -1).T
18
19     # Matrix multiplication
20     Z = cols @ W_col + b
21
22     pad=0
23
24     # Kembalikan ke bentuk feature map
25     H_out = (H + 2 * pad - KH) // stride + 1
26     W_out = (W_input + 2 * pad - KW) // stride + 1
27
28     Z = Z.reshape(
29         N,
30         H_out,
31         W_out,
32         F
33     )
34

```

```

35     Z = Z.transpose(0, 3, 1, 2) # (N, H_out, W_out, F)
transpose(0,3,1,2) -> (N, F, H_out, W_out)
36
37     conv_cache = (
38         x_pad,
39         W,
40         stride,
41         cols,
42         x.shape
43     )
44
45     return Z, conv_cache
46
47 def maxpool_forward(x, size=2, stride=2):
48     """
49     x : input gambar
50
51     Max pooling forward. Nilai yang diambil hanya nilai yang
52     terbesar dari kernel
53     """
54     n, c, h, width = x.shape
55
56     h_out = (h - size) // stride + 1 # karena channel yang
57     dipakai hanya 1, berarti 1.1 = 1
58     w_out = (width - size) // stride + 1
59
60     out = np.empty((n, c, h_out, w_out), dtype=x.dtype)
61
62     for i in range(h_out):
63         for j in range(w_out):
64             h_start = i * stride
65             w_start = j * stride
66
67             window = x[
68                 :,
69                 :,
70                 h_start:h_start + size,
71                 w_start:w_start + size
72             ] # window ini hanya yang dimodif height dan
73             weight + size...
74
75             out[:, :, i, j] = window.max(axis=(2, 3)) # :, :
76             -> 1. Input Image, 2. Input Channel

```

```

74     # Simpan nilai yang dibutuhkan saat backward pass (posisi
      window
75     # dipakai untuk menentukan ke mana gradient harus
      dialirkan kembali)
76     pool_cache = (x, size, stride)
77
78     return out, pool_cache
79
80
81 def flatten_forward(x):
82     """
83     Ubah feature map 4D menjadi 2D untuk dense layer.
84     """
85     return x.reshape(x.shape[0], -1), x.shape

```

Listing 3: Forward pass layer convolution

Variabel `conv_cache` pada kode di atas menyimpan nilai-nilai yang dihasilkan saat forward pass (seperti input dan hasil `im2col`), yang nantinya dibutuhkan lagi saat menghitung gradien pada tahap backpropagation (lihat STEP 8), supaya tidak perlu dihitung ulang dari nol

STEP 3: Pooling

Pooling merupakan proses untuk mengurangi dimensi dari feature map/output (panjang dan lebar), namun menjaga informasi penting dari feature map tersebut. Formula untuk ukuran output dari proses pooling adalah

$$h_{\text{out}}, w_{\text{out}} = \frac{n_h - f}{s} + 1 \times n_c \quad (4)$$

yang dimana, n_h adalah tinggi input, n_w adalah lebar input, f adalah ukuran filter, s adalah panjang stride, dan n_c adalah jumlah channel. Untuk contoh, jika memiliki feature map dengan ukuran 4×4 , filter dengan ukuran 2×2 atau biasanya ditulis 2 dan stride 2×2 , maka:

$$\frac{4 - 2}{2} + 1 = 2 \quad (5)$$

output dari proses pooling adalah 2×2 , dengan channel yang sama dengan input. Pooling yang digunakan pada proyek ini adalah max pooling, yaitu mengambil nilai maksimum dari setiap filter. Untuk ilustrasi proses pooling bisa dilihat pada gambar berikut.

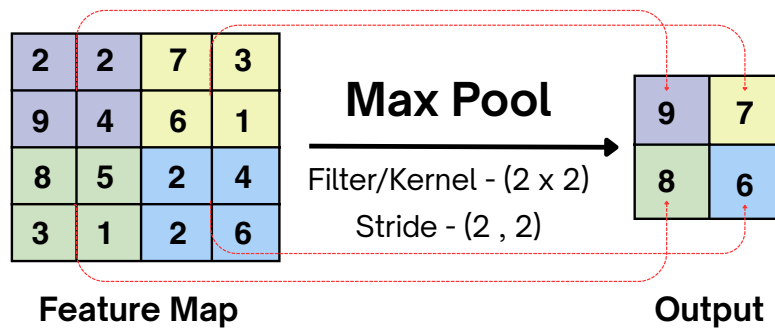


Figure 4: Ilustrasi proses max pooling pada feature map

Pooling biasanya disebut sebagai detektor fitur. Berikut implementasi max pooling forward yang digunakan pada proyek ini:

```

1 def maxpool_forward(x, size=2, stride=2):
2     """
3     x : input gambar
4
5     Max pooling forward. Nilai yang diambil hanya nilai yang
6     terbesar dari kernel
7     """
8     n, c, h, width = x.shape
9
10    h_out = (h - size) // stride + 1 # karena channel yang
11    dipakai hanya 1, berarti 1.1 = 1
12    w_out = (width - size) // stride + 1
13
14    out = np.empty((n, c, h_out, w_out), dtype=x.dtype)
15
16    for i in range(h_out):
17        for j in range(w_out):
18            h_start = i * stride
19            w_start = j * stride
20
21            window = x[
22                :,
23                :,
24                h_start:h_start + size,
25                w_start:w_start + size
26            ] # window ini hanya yang dimodif height dan
27            weight + size...
28
29            out[:, :, i, j] = window.max(axis=(2, 3)) # :, :

```

```

27     -> 1. Input Image, 2. Input Channel
28     # Simpan nilai yang dibutuhkan saat backward pass (posisi
29     # dipakai untuk menentukan ke mana gradient harus
    dialirkan kembali)
30     pool_cache = (x, size, stride)
31
32     return out, pool_cache

```

Listing 4: Forward pass max pooling

Sama seperti conv2d forward, di sini juga disimpan pool cache yang berisi input dan posisi window, karena saat backpropagation kita perlu tahu piksel mana yang menghasilkan nilai maksimum agar gradien bisa dialirkan ke posisi yang tepat.

STEP 4: Flatten

Flatten merupakan proses untuk mengubah data yang memiliki dimensi besar direduksi. Misal, jika memiliki data dengan 2 dimensi, maka setelah proses flattening, data tersebut akan menjadi 1 dimensi. Proses flattening ini dilakukan agar data dapat diteruskan ke fully connected layer.

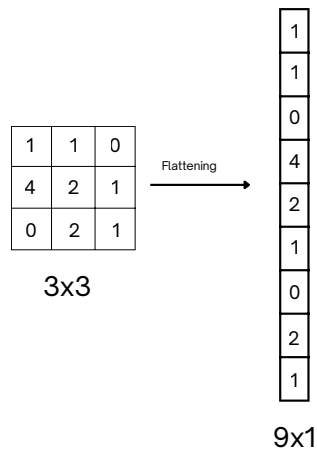


Figure 5: Ilustrasi proses flatten

STEP 5: Fully Connected

Fully connected atau dense layer merupakan kumpulan neuron yang saling terhubung dengan semua neuron pada layer sebelumnya. Satu neuron biasanya kita sebut sebagai perceptron. Perceptron ini memiliki bobot dan bias. Untuk ilustrasi perceptron bisa dilihat pada gambar dibawah ini.

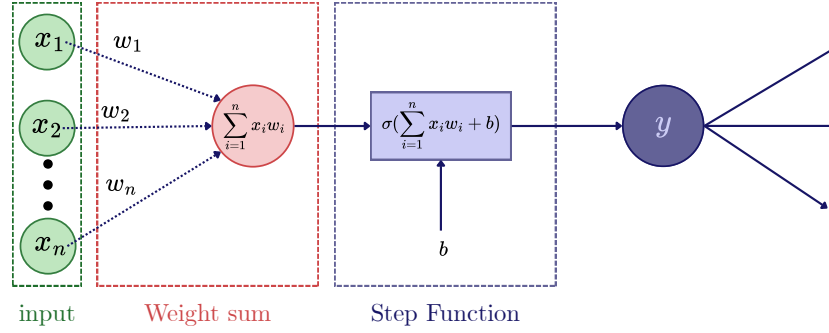


Figure 6: Ilustrasi perceptron

Dari proses perceptron ini, apabila kita memiliki banyak neuron, maka kita bisa menyebutnya sebagai fully connected layer. Secara rumus matematis, bisa ditulis sebagai:

$$A^{(l)} = \sigma^{(l)}(W^{(l)} A^{(l-1)} + b^{(l)}), l = 1, 2, \dots, L \quad (6)$$

dengan $A^{(0)} = X$ sebagai input, $W^{(l)}$ adalah matriks bobot pada lapisan ke- l , $b^{(l)}$ adalah vektor bias pada lapisan ke- l , $\sigma^{(l)}$ adalah fungsi aktivasi non-linear pada lapisan ke- l , dan L adalah jumlah total lapisan dalam jaringan.

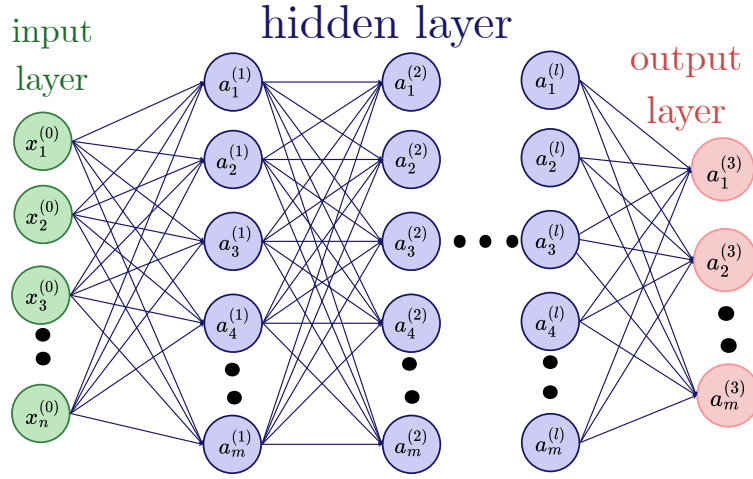


Figure 7: Ilustrasi arsitektur neural network dengan beberapa lapisan fully connected

STEP 6: Softmax

Softmax merupakan fungsi aktivasi yang mengubah ke distribusi probabilitas. Maksudnya adalah output dari softmax akan memiliki nilai antara 0 dan 1, dan jumlah dari semua output softmax adalah 1. Rumus dari softmax adalah:

$$\sigma(z_i) = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}} \quad (7)$$

Pada proyek ini, softmax digunakan pada output layer fully connected layer, karena cocok digunakan untuk klasifikasi. Softmax ini akan mengubah output model yang masih berupa nilai mentah atau *logits* menjadi probabilitas, sehingga dapat digunakan untuk menentukan kelas dari input data. Perlu diingat bahwa softmax bukan loss function. Softmax menghasilkan probabilitas, sedangkan cross-entropy pada STEP 7 mengukur seberapa jauh probabilitas tersebut dari label yang benar.

STEP 7: Loss Function

Loss function merupakan fungsi yang digunakan untuk mengukur seberapa baik model dalam memprediksi output. Pada proyek ini, loss function yang

digunakan adalah categorical cross-entropy, karena MNIST memiliki 10 kelas, yaitu digit 0 sampai 9. Jadi, loss yang digunakan bukan binary cross-entropy. Untuk satu batch yang berisi N sampel dan C kelas, rumusnya adalah:

$$L = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{i,c} \log(p_{i,c}) \quad (8)$$

yang dimana:

- N adalah jumlah sampel dalam satu batch,
- C adalah jumlah kelas, pada projek ini $C = 10$,
- $y_{i,c}$ bernilai 1 apabila sampel ke- i termasuk kelas c , dan 0 untuk kelas lainnya,
- $p_{i,c}$ adalah probabilitas hasil softmax untuk sampel ke- i pada kelas c .

Karena hanya kelas yang benar yang memiliki nilai $y_{i,c} = 1$, implementasinya cukup mengambil $-\log(p)$ pada indeks label yang benar, kemudian dirata-ratakan. Sebelum menghitung eksponensial, setiap logits dikurangi nilai maksimum pada baris tersebut. Pengurangan ini tidak mengubah hasil probabilitas softmax, tetapi membuat perhitungan lebih stabil supaya nilai eksponensial tidak terlalu besar.

```

1 def softmax_cross_entropy(z, y):
2     z = z - np.max(z, axis=1, keepdims=True)
3
4     exp_z = np.exp(z)
5     probs = exp_z / np.sum(exp_z, axis=1, keepdims=True)
6
7     n = z.shape[0]
8     loss = -np.log(probs[np.arange(n), y] + 1e-12).mean()
9
10    dz = probs.copy()
11    dz[np.arange(n), y] -= 1
12    dz /= n
13
14    return loss, dz

```

Listing 5: Softmax dan categorical cross-entropy

Ketika softmax dan categorical cross-entropy digabungkan, gradien loss terhadap logits menjadi lebih sederhana:

$$\frac{\partial L}{\partial z} = \frac{p - y}{N}. \quad (9)$$

Nilai gradien ini disimpan pada variabel $\mathbf{dz}$ dan menjadi titik awal proses backpropagation pada STEP 8 [1].

STEP 8: Backpropagation

Pada forward pass, data bergerak dari input menuju output sampai menghasilkan loss. Namun, model belum mengetahui bobot mana yang harus diperbaiki. Backpropagation digunakan untuk menghitung seberapa besar pengaruh setiap bobot dan bias terhadap loss dengan menelusuri jaringan dari output kembali ke input. Ide ini menggunakan *chain rule* dari kalkulus [3].

Sebagai contoh sederhana, apabila loss L dipengaruhi oleh z , sedangkan z dipengaruhi oleh x , maka:

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial z} \frac{\partial z}{\partial x}. \quad (10)$$

Artinya, gradien dari layer setelahnya dikalikan dengan turunan lokal dari operasi yang sedang dilewati. Inilah alasan nilai-nilai pada forward pass disimpan ke dalam `layer_caches`: nilai tersebut dapat digunakan kembali tanpa menghitung forward pass dari awal.

Backward pada Fully Connected dan ReLU

Jika operasi pada fully connected layer ditulis sebagai $Z = AW + b$, maka gradiennya adalah:

$$dW = A^T dZ, \quad (11)$$

$$db = \sum_{i=1}^N dZ_i, \quad (12)$$

$$dA = dZW^T. \quad (13)$$

Urutan transpose pada rumus tersebut penting supaya shape matriks hasilnya sama dengan shape parameter yang akan diperbarui. Sementara itu, ReLU hanya meneruskan gradien jika nilai Z pada forward pass lebih besar dari nol:

$$dZ = dA \odot \mathcal{K}(Z > 0). \quad (14)$$

Simbol $\odot$ berarti perkalian elemen demi elemen dan $\mathcal{K}$ merupakan mask yang bernilai 1 ketika kondisi benar.

Backward pada Flatten dan Max Pooling

Flatten tidak mengubah nilai, tetapi hanya mengubah shape. Oleh karena itu, backward flatten cukup mengembalikan gradien ke shape sebelum flatten. Pada max pooling, gradien hanya dikirim ke posisi yang menghasilkan nilai maksimum pada setiap window. Posisi lainnya mendapat gradien nol.

```

1 def maxpool_backward(dout, pool_cache):
2     x, size, stride = pool_cache
3     n, c, h, width = x.shape
4     h_out, w_out = dout.shape[2], dout.shape[3]
5
6     dx = np.zeros_like(x)
7
8     for i in range(h_out):
9         for j in range(w_out):
10             h_start = i * stride
11             w_start = j * stride
12
13             window = x[:, :, h_start:h_start + size,
14                        w_start:w_start + size]
15             window_max = window.max(
16                 axis=(2, 3), keepdims=True
17             )
18             mask = (window == window_max)
19
20             dx[:, :, h_start:h_start + size,
21                w_start:w_start + size] += (
22                 mask * dout[:, :, i, j][:, :, None, None]
23             )
24
25     return dx
26
27 def flatten_backward(dout, shape):

```

```
28     return dout.reshape(shape)
```

Listing 6: Backward max pooling dan flatten

Apabila ada dua nilai maksimum yang sama di dalam satu window, implementasi di atas akan mengalirkan gradien ke seluruh posisi yang sama-sama maksimum. Hal ini terjadi karena seluruh posisi tersebut bernilai `True` pada `mask`.

Backward pada Convolution

Pada forward convolution, patch input sudah diubah menjadi matriks `cols`. Misalkan dZ_{col} adalah gradien output convolution yang sudah diratakan, maka:

$$dW_{\text{col}} = \text{cols}^T dZ_{\text{col}}, \quad (15)$$

$$db = \sum dZ_{\text{col}}, \quad (16)$$

$$d\text{cols} = dZ_{\text{col}} W_{\text{col}}^T. \quad (17)$$

Setelah mendapatkan $d\text{cols}$, setiap patch dikembalikan ke posisi asalnya pada input. Jika beberapa patch bertumpuk pada piksel yang sama, gradiennya dijumlahkan.

```
1 def conv2d_backward(dz, conv_cache):
2     x_pad, w, stride, cols, x_shape = conv_cache
3     n, c, h, width = x_shape
4     f, _, kh, kw = w.shape
5     h_out, w_out = dz.shape[2], dz.shape[3]
6
7     w_col = w.reshape(f, -1).T
8     dz_reshaped = dz.transpose(0, 2, 3, 1).reshape(
9         n * h_out * w_out, f
10    )
11
12    dw_col = cols.T @ dz_reshaped
13    dw = dw_col.T.reshape(w.shape)
14    db = dz_reshaped.sum(axis=0)
15
16    dcols = dz_reshaped @ w_col.T
17    dcols = dcols.reshape(n, h_out, w_out, c, kh, kw)
18    dx_pad = np.zeros_like(x_pad)
19
20    for i in range(kh):
```

```

21         for j in range(kw):
22             patch = dcols[:, :, :, :, i, j]
23             patch = patch.transpose(0, 3, 1, 2)
24             dx_pad[:, :,
25                     i:i + stride * h_out:stride,
26                     j:j + stride * w_out:stride] += patch
27
28     return dx_pad, dw, db

```

Listing 7: Backward layer convolution

Semua backward layer tersebut kemudian disusun dengan urutan yang berlawanan dari forward pass: output dense → ReLU dense → dense pertama → flatten → max pooling → ReLU convolution → convolution.

```

1 def backward_cnn(dz3, layer_caches, params):
2     W1, b1, W2, b2, W3, b3 = params
3
4     # Output dense
5     A2, W3 = layer_caches['dense3']
6     dW3 = A2.T @ dz3
7     db3 = dz3.sum(axis=0)
8     dA2 = dz3 @ W3.T
9
10    # ReLU dan dense pertama
11    dz2 = dA2 * (layer_caches['relu2'] > 0)
12    F, W2 = layer_caches['dense2']
13    dW2 = F.T @ dz2
14    db2 = dz2.sum(axis=0)
15    dF = dz2 @ W2.T
16
17    # Flatten, max pooling, dan ReLU convolution
18    dP1 = flatten_backward(dF, layer_caches['flat'])
19    dA1 = maxpool_backward(dP1, layer_caches['pool1'])
20    dz1 = dA1 * (layer_caches['relu1'] > 0)
21
22    # Convolution
23    dX, dW1, db1 = conv2d_backward(
24        dz1, layer_caches['conv1']
25    )
26
27    return dW1, db1, dW2, db2, dW3, db3

```

Listing 8: Urutan backpropagation seluruh CNN

Hasil fungsi ini masih berupa gradien. Parameter model baru benar-benar berubah ketika gradien tersebut digunakan pada tahap optimasi berikutnya.

STEP 9: Gradient Descent

Backpropagation dan gradient descent memiliki tugas yang berbeda. Backpropagation menghitung gradien, sedangkan gradient descent menggunakan gradien tersebut untuk memperbarui parameter supaya loss bergerak ke arah yang lebih kecil. Untuk setiap parameter θ , proses update ditulis sebagai:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} L, \quad (18)$$

yang dimana η adalah learning rate. Tanda minus digunakan karena gradien menunjukkan arah kenaikan paling cepat, sedangkan tujuan training adalah menurunkan loss [1].

Sebelum melihat implementasinya, ada tiga hyperparameter penting:

- **Epoch** adalah satu kali proses ketika seluruh data training sudah dilewati. Epoch terlalu sedikit dapat membuat model belum selesai belajar, sedangkan terlalu banyak dapat meningkatkan risiko overfitting.
- **Batch size** adalah jumlah sampel yang diproses sebelum parameter diperbarui. Batch kecil menghasilkan update lebih sering dan lebih berisik, sedangkan batch besar membutuhkan memori lebih banyak tetapi perhitungannya biasanya lebih stabil.
- **Learning rate** menentukan besar langkah update. Nilai terlalu kecil membuat training lambat, sedangkan nilai terlalu besar dapat membuat loss naik-turun atau bahkan gagal konvergen.

Pada awal setiap epoch, indeks data diacak menggunakan fungsi `permutation` dari NumPy. Setelah satu batch menyelesaikan forward pass dan backward pass, seluruh bobot dan bias diperbarui menggunakan learning rate yang sama.

```
1 def train_cnn_with_history(  
2     X, y, X_val, y_val,  
3     epochs=1, batch_size=128, lr=0.01  
4 ):  
5     params = init_parameters_cnn()  
6     n = X.shape[0]  
7     history = {  
8         'train_loss': [], 'train_acc': [],  
9         'val_loss': [], 'val_acc': [],  
10        'batch_loss': [],
```

```

11     }
12
13     for epoch in range(1, epochs + 1):
14         idx = np.random.permutation(n)
15         X_sh, y_sh = X[idx], y[idx]
16         losses, correct, total = [], 0, 0
17
18         for start in range(0, n, batch_size):
19             xb = X_sh[start:start + batch_size]
20             yb = y_sh[start:start + batch_size]
21
22             logits, layer_caches = forward_cnn(xb, params)
23             loss, dz3 = softmax_cross_entropy(logits, yb)
24             losses.append(loss)
25             history['batch_loss'].append(float(loss))
26             predictions = np.argmax(logits, axis=1)
27             correct += np.sum(predictions == yb)
28             total += yb.shape[0]
29
30             gradients = backward_cnn(
31                 dz3, layer_caches, params
32             )
33             dW1, db1, dW2, db2, dW3, db3 = gradients
34             W1, b1, W2, b2, W3, b3 = params
35
36             W1 -= lr * dW1; b1 -= lr * db1
37             W2 -= lr * dW2; b2 -= lr * db2
38             W3 -= lr * dW3; b3 -= lr * db3
39             params = (W1, b1, W2, b2, W3, b3)
40
41             train_loss = np.mean(losses)
42             train_acc = correct / total
43             val_loss, val_acc = evaluate_cnn(
44                 X_val, y_val, params
45             )
46
47             history['train_loss'].append(train_loss)
48             history['train_acc'].append(train_acc)
49             history['val_loss'].append(val_loss)
50             history['val_acc'].append(val_acc)
51
52     return params, history

```

Listing 9: Training CNN menggunakan mini-batch gradient descent

Eksperimen pada notebook menggunakan 5 epoch, batch size 512, dan

learning rate 0.01. Nilai tersebut bukan aturan tetap. Hyperparameter dapat diubah dan dibandingkan dengan melihat apakah loss turun secara stabil serta apakah akurasi pada data yang tidak digunakan untuk update ikut membaik.

STEP 10: Evaluasi Model

Setelah parameter diperbarui, model perlu dievaluasi untuk melihat apakah prediksinya membaik. Fungsi berikut membagi data evaluasi menjadi beberapa batch supaya seluruh data tidak harus dimasukkan ke memori pada waktu yang sama.

```
1 def evaluate_cnn(X, y, params, batch_size=512):
2     total_loss = 0.0
3     correct = 0
4     total = 0
5
6     for start in range(0, X.shape[0], batch_size):
7         xb = X[start:start + batch_size]
8         yb = y[start:start + batch_size]
9
10        logits, _ = forward_cnn(xb, params)
11        loss, _ = softmax_cross_entropy(logits, yb)
12
13        bs = yb.shape[0]
14        total_loss += loss * bs
15        predictions = np.argmax(logits, axis=1)
16        correct += np.sum(predictions == yb)
17        total += bs
18
19    return total_loss / total, correct / total
```

Listing 10: Menghitung loss dan accuracy model

Loss setiap batch dikalikan kembali dengan jumlah sampel pada batch tersebut sebelum dijumlahkan. Hal ini penting karena batch terakhir belum tentu memiliki ukuran yang sama. Accuracy dihitung dengan rumus:

$$\text{accuracy} = \frac{\text{jumlah prediksi benar}}{\text{jumlah seluruh sampel}}. \quad (19)$$

Kelas prediksi diambil dari indeks logits terbesar menggunakan `argmax`. Softmax tidak perlu dihitung lagi untuk menentukan kelas, karena softmax mempertahankan urutan nilai logits.

Hasil Training

Berikut hasil yang tersimpan pada output notebook ketika model dilatih selama 5 epoch.

Epoch	Train Loss	Train Accuracy	Eval. Loss	Eval. Accuracy
1	1.5544	57.71%	0.8842	80.15%
2	0.6626	83.73%	0.5043	86.79%
3	0.4622	87.32%	0.3994	89.18%
4	0.3899	89.07%	0.3492	90.26%
5	0.3506	90.04%	0.3190	91.00%

Table 4: Loss dan accuracy untuk setiap epoch

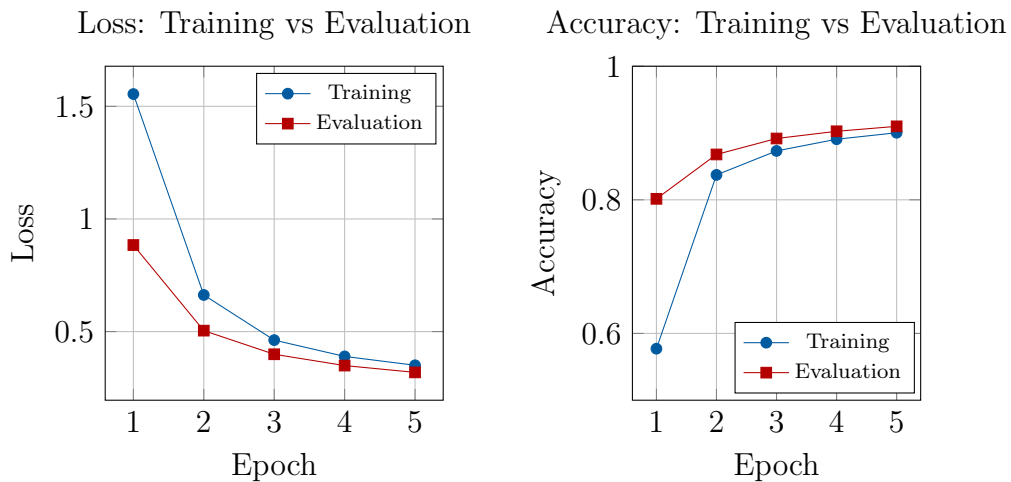


Figure 8: Perubahan loss dan accuracy selama training

Dari tabel dan grafik, train loss turun dari 1.5544 menjadi 0.3506, sedangkan evaluation loss turun dari 0.8842 menjadi 0.3190. Pada saat yang sama, train accuracy naik menjadi 90.04% dan evaluation accuracy menjadi 91.00%. Pola ini menunjukkan bahwa parameter yang diperbarui oleh gradient descent membuat prediksi model semakin baik pada lima epoch tersebut. Namun, hasil ini belum cukup untuk menyimpulkan bahwa konfigurasi tersebut adalah konfigurasi terbaik; eksperimen dengan epoch, batch size, dan learning rate lain tetap diperlukan.

Catatan tentang data evaluasi

Di notebook, fungsi training menerima nama parameter `X_val` dan `y_val`, tetapi argumen yang diberikan sebenarnya adalah `X_test` dan `y_test`. Artinya, test set dilihat pada setiap epoch. Hasil di atas tetap ditampilkan sesuai notebook, tetapi untuk eksperimen yang lebih ketat sebaiknya data dibagi menjadi training, validation, dan test. Validation set digunakan untuk memilih hyperparameter, sedangkan test set digunakan satu kali setelah seluruh keputusan training selesai. Pembagian ini membantu menjaga evaluasi akhir tetap netral [1].

Implementasi CNN dari awal ini memperlihatkan hubungan lengkap antara forward pass, loss, backpropagation, gradient descent, dan evaluasi. Pendekatan berbasis gradien yang sama juga menjadi dasar penggunaan CNN pada pengenalan digit tulisan tangan [4].

References

- [1] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press. [link](#).
- [2] Chitta Ranjan, S., & Saha, S. (2020). *Understanding Deep Learning Application in Rare Event Prediction*. Springer.
- [3] Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). *Learning Representations by Back-Propagating Errors*. *Nature*, 323, 533–536. doi:10.1038/323533a0.
- [4] LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). *Gradient-Based Learning Applied to Document Recognition*. *Proceedings of the IEEE*, 86(11), 2278–2324. doi:10.1109/5.726791.